ECE Paris · Rapport de projet

AutoLoc API

Application de gestion de location de véhicules
Spring Boot · qualité SonarQube · CI/CD Jenkins

Élément	Valeur
Étudiant	Abdouljalil Diaby
Établissement	ECE Paris
Date	9 novembre 2026
Dépôt Git	github.com/DAJBINARY/autoloc-api (branche main)
Image Docker	hub.docker.com/r/dajbinary/autoloc-api
Technologies	Java 17 · Spring Boot 4.1.2 · Spring MVC / Thymeleaf · Spring Data JPA · MySQL 8 · Spring Security · Prometheus / Grafana · SonarQube · Jenkins · Docker

283	91,4 %	0	A	7/7
tests automatisés	couverture de code	bug · 0 vulnérabilité	notes SonarQube	étapes CI au vert

Sommaire

1. Présentation du projet 3

2. Architecture technique 4

3. Modèle de données 5

4. Fonctionnalités et écrans 6

5. Stratégie de tests 7

6. Monitoring 8

7. Qualité du code avec SonarQube 9

8. Pipeline CI/CD avec Jenkins 10

9. Bilan et perspectives 12

1. Présentation du projet

AutoLoc est une application web de gestion d'une agence de location de véhicules. Elle couvre trois publics, chacun avec ses écrans et ses droits :

- **les visiteurs**, sur le site public : catalogue des véhicules filtrable par dates, catégorie et agence, fiche de chaque véhicule avec estimation du prix, inscription ;
- **les clients**, dans leur espace : réservation en ligne, paiement par carte (simulé), suivi et annulation des réservations, contrat imprimable, plan du parking pour retrouver le véhicule, profil ;
- **le personnel** (agents et managers), dans le back office : gestion complète des véhicules, clients, agences, employés, réservations, contrats, paiements, maintenances et équipements, planning, disponibilités, plan du parking, statistiques et tableau de bord.

Le projet a été mené par étapes successives, chacune correspondant à un commit sur la branche *main* du dépôt GitHub :

Date	Étape
04/11/2026	Workshop 1 : initialisation Spring Boot, entité Vehicule, profil *dev* et données de démo
04/11/2026	Préparation du Workshop 2 : entités restantes (Agence, Client, Employé, Réservation, Contrat…)
05/11/2026	Ajout de Spring Boot Actuator
05/11/2026	Écrans MVC (back office), site public et parcours de réservation client
09/11/2026	Monitoring : Prometheus et tableau de bord Grafana
09/11/2026	Corrections des remontées SonarQube for IDE
09/11/2026	Pipeline CI/CD Jenkins, puis corrections réseau trouvées lors des premiers builds

2. Architecture technique

L'application suit une architecture en couches classique de Spring Boot. Chaque couche ne dépend que de celle du dessous : les contrôleurs n'accèdent jamais directement aux repositories, toutes les règles métier et les transactions sont dans les services.

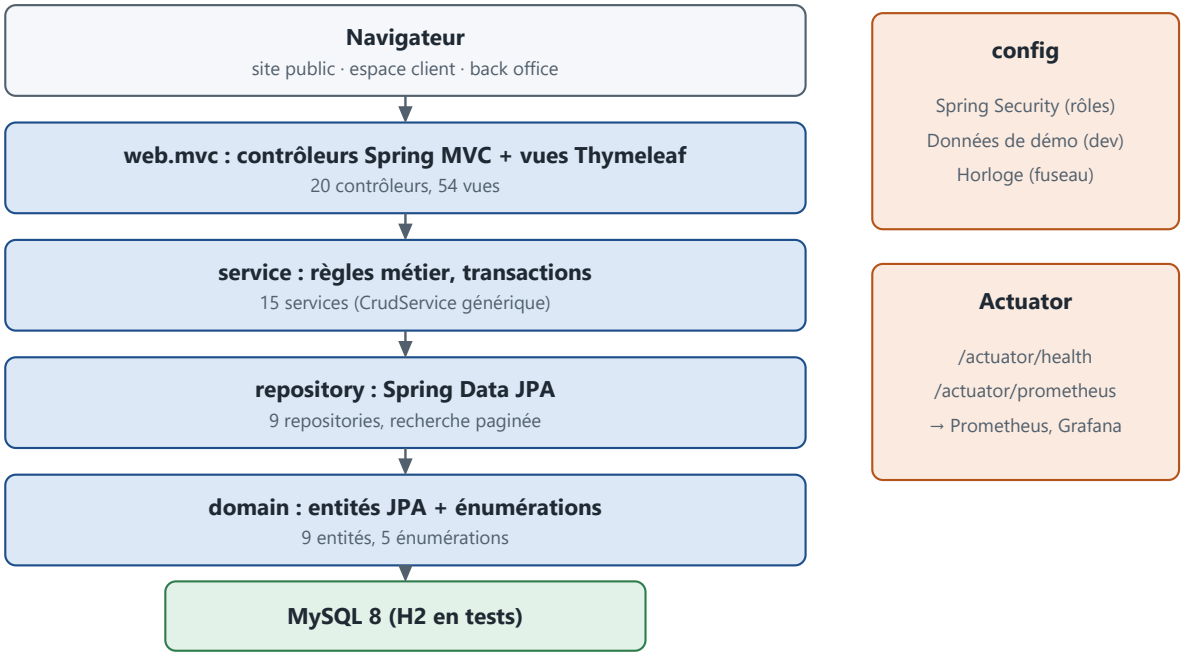


Figure 1 – Couches de l'application (package fr.ece.autoloc)

Brique	Rôle dans le projet
Spring Boot 4.1.2, Java 17	Socle de l'application, configuration automatique, serveur web embarqué
Spring MVC + Thymeleaf	Rendu des pages côté serveur, formulaires validés (Bean Validation)
Spring Data JPA / Hibernate	Accès aux données ; le schéma est créé et mis à jour par Hibernate
MySQL 8	Base de données de l'application (H2 en mémoire pour les tests)
Spring Security	Connexion par formulaire, rôles CLIENT, AGENT et MANAGER, mots de passe hachés
Actuator + Micrometer	Santé de l'application et métriques au format Prometheus

Choix de conception notables

- **Un CRUD générique** : `CrudService<T>` et `CrudController<T>` factorisent liste paginée avec recherche, fiche, création, modification et suppression pour les 9 entités. Chaque contrôleur ne redéfinit que ses particularités (règles de validation croisées, actions de workflow, refus de suppression).
- **Sécurité des formulaires** : l'identifiant vient toujours de l'URL, jamais du formulaire, et les entités liées ne peuvent pas être modifiées par un paramètre du type *client.nom=…*.
- **Isolation des clients** : la réservation d'un autre client répond exactement comme une réservation inexistante (404), pour ne rien laisser deviner.
- **Fuseau horaire explicite** : toutes les dates « du jour » utilisent Europe/Paris, quel que soit le fuseau du serveur ou du conteneur.

3. Modèle de données

Le domaine comporte 9 entités JPA. Les associations ont été ajoutées pour les écrans MVC ; les relations sont chargées à la demande (LAZY) pour éviter les requêtes inutiles.

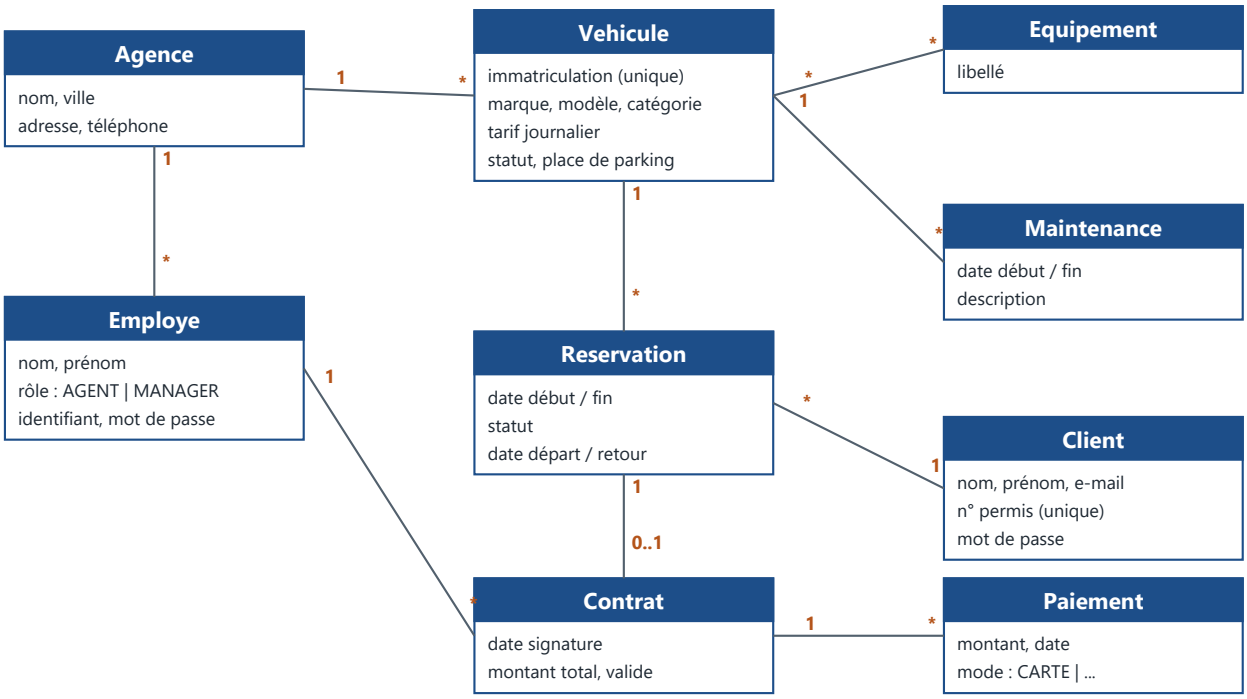


Figure 2 – Entités et associations (cardinalités en orange)

Énumération	Valeurs
StatutVehicule	Disponible, Loué, En maintenance
StatutReservation	En attente, Confirmée, Annulée, Terminée
CategorieVehicule	Citadine, Berline, SUV, Utilitaire
RoleEmploye	Agent, Manager
ModePaiement	Carte, Espèces, Virement

Règles d'intégrité garanties par la base

- immatriculation d'un véhicule, e-mail et numéro de permis d'un client uniques ;
- une place de parking ne peut être occupée que par un seul véhicule dans une même agence ;
- un contrat au plus par réservation ;
- les règles métier (chevauchement de réservations, véhicule en maintenance, location en cours non modifiable, montant des paiements) sont vérifiées par les services et affichées dans les formulaires.

4. Fonctionnalités et écrans

L'interface compte 54 pages Thymeleaf, réparties en trois espaces.

Espace	Écrans et actions
Site public	Accueil avec véhicules en vitrine · catalogue avec recherche, filtres (dates, catégorie, agence) et pagination · fiche véhicule avec disponibilité et prix estimé · inscription avec connexion automatique
Espace client (/espace)	Tableau de bord et prochaine location · nouvelle réservation · paiement en ligne (numéro de carte contrôlé par l'algorithme de Luhn, date d'expiration, cryptogramme) ou paiement en agence · confirmation · contrat imprimable · annulation · plan du parking · profil et mot de passe
Back office	CRUD des 9 entités · workflow des réservations (confirmer, annuler, départ, retour) · clôture des maintenances · impression des contrats · planning mensuel · recherche de disponibilités avec prix · plan du parking · statistiques (managers) · tableau de bord avec indicateurs clés
Compte	Connexion, création du premier compte manager au premier lancement, changement de mot de passe

Droits d'accès

Rôle	Accès
Visiteur	Site public, catalogue, inscription, connexion, santé et métriques de l'application
CLIENT	Uniquement son espace /espace et ses propres réservations
AGENT	Back office, sauf suppression, statistiques, gestion des agences et des employés
MANAGER	Tout le back office, dont suppressions, statistiques, agences et employés

Un utilisateur connecté est redirigé vers son espace selon son rôle. Un employé ne peut pas supprimer son propre compte, et une réservation dont la location est en cours ne peut pas être supprimée.

5. Stratégie de tests

Le projet compte **283 tests automatisés**, organisés en pyramide. Les tests s'exécutent sur une base H2 en mémoire, ce qui les rend rapides et indépendants de la machine, sauf MySqlConnexionTest qui valide le fonctionnement sur un vrai serveur MySQL dans le pipeline.

Niveau	Classes de test	Tests
Unitaires – domaine	DomaineTest, ValidationDomaineTest	35
Unitaires – services	Réservation, ReservationClient, Catalogue, Compte, Maintenance, Planification, Statistiques, CrudService	99
Unitaires – web	PaiementFormTest (carte bancaire)	27
Intégration	RepositoriesIntegrationTest, ServicesIntegrationTest (base H2)	43
Intégration MySQL	MySqlConnexionTest (exécuté dans le pipeline Jenkins)	2
Écrans (MockMvc)	IhmIntegration, SiteClient, Sécurité, EcransMetier, DemoData	68
Fonctionnels	Parcours complet de location, workflow Grafana	8
Contexte	Démarrage de l'application	1
Total		**283**

91,4 %	95,8 %	81,2 %	0
couverture globale	couverture des lignes	couverture des branches	test en échec

La couverture est mesurée par JaCoCo pendant `mvn verify` et transmise à SonarQube. Les tests de sécurité vérifient pour chaque rôle les pages accessibles et refusées ; le test fonctionnel déroule le parcours réel d'un client, de l'inscription au paiement, sur un serveur démarré.

6. Monitoring

L'application expose ses métriques avec Spring Boot Actuator et Micrometer sur `/actuator/prometheus`. Une pile Docker Compose démarre Prometheus, qui collecte ces métriques, et Grafana, qui les affiche dans un tableau de bord provisionné automatiquement.

Service	Adresse	Rôle
Application	http://localhost:8080	Métriques et état de santé via Actuator
Prometheus	http://localhost:19090	Collecte et stockage des métriques
Grafana	http://localhost:3000	Tableau de bord « AutoLoc API »

Indicateurs du tableau de bord Grafana

Domaine	Panneaux
Disponibilité	État de l'application, temps de fonctionnement
Trafic	Requêtes par seconde, requêtes par page, réponses par code HTTP, erreurs serveur (5xx)
Performance	Temps de réponse au 95e centile, connexions à la base de données
JVM	Mémoire (heap), utilisation du processeur, threads, temps passé dans le ramasse-miettes

> Le port 9090 de Prometheus étant dans une plage réservée par Windows sur la machine, Prometheus est publié sur le port 19090. Le calcul des centiles de temps de réponse est activé dans `application.properties`.

7. Qualité du code avec SonarQube

La qualité est contrôlée à deux niveaux : **SonarQube for IDE** (plugin IntelliJ) pendant le développement, puis un **serveur SonarQube** qui analyse chaque build du pipeline et applique un Quality Gate.

Corrections issues de SonarQube for IDE

Chaque règle signalée sur un fichier a été corrigée dans tout le projet en une fois, plutôt que fichier par fichier :

Règle	Problème	Correction
S8688	now() sans fuseau horaire	Constante partagée Horloge.ZONE (Europe/Paris)
S1192	Littéraux dupliqués	Classe Attributs pour les noms partagés, constantes locales ailleurs
S107	Constructeurs à 9 paramètres	Injection d'une liste de services ; EspaceClientController découpé en 3 contrôleurs ; constructeur de Vehicule réduit
S6809	@Transactional appelé via this	Appel direct du repository dans PaiementService et CompteService
S1172 / S4276	Paramètre inutilisé, interface fonctionnelle	Crochet verifierSuppression ; LongConsumer
S3358	Ternaire imbriqué	Méthode montant() explicite
S112 / S1130	throws Exception générique	Déclaration retirée (Spring Security 7)
S6437	Mot de passe en dur	Mot de passe de démo lu depuis application-dev.properties
S1128	Import inutilisé	Supprimé

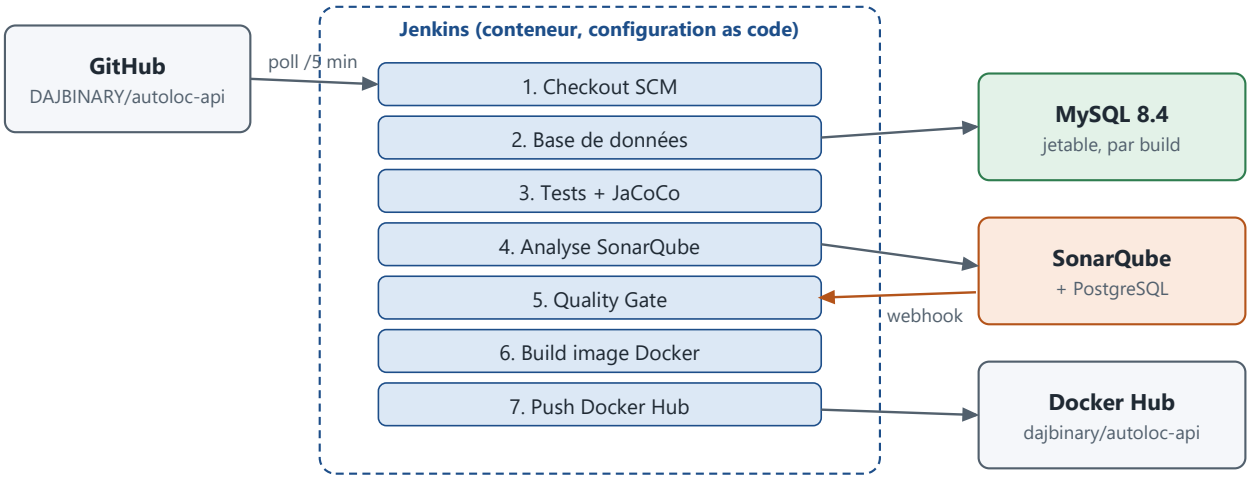
Résultat de l'analyse sur le serveur SonarQube

Mesure	Valeur	Mesure	Valeur
Quality Gate	**Passed**	Lignes de code	3 534
Bugs	0 (note A)	Classes / méthodes	84 / 290
Vulnérabilités	0 (note A)	Couverture	91,4 %
Security hotspots	0	Duplication	0,0 %
Code smells	1 (note A)	Dette technique	20 min

La seule remarque restante (règle S8786) concerne une expression régulière d'un test (WorkflowGrafanaTest), et non le code de l'application.

8. Pipeline CI/CD avec Jenkins

Le pipeline demandé enchaîne : récupération du code (SCM), connexion à une base de données, tests, analyse SonarQube, construction de l'image Docker et publication sur Docker Hub. Toute l'infrastructure est décrite dans le dépôt et se lance avec une seule commande.



Réseau Docker « autoloc-ci » · Docker de l'hôte via /var/run/docker.sock

Figure 3 – Architecture du pipeline

Étape	Ce qu'elle fait	Durée*
Checkout SCM	Récupère le commit de la branche main depuis GitHub	1 s
Base de données	Démarre un MySQL 8.4 jetable sur le réseau CI, attend qu'il soit prêt, puis démarre l'application dessus et vérifie la connexion et la création des tables	43 s
Tests	mvn verify : 283 tests, rapport de couverture JaCoCo, construction du jar ; résultats publiés dans Jenkins et jar archivé	62 s
SonarQube	Analyse du code et de la couverture, envoyée au serveur SonarQube	38 s
Quality Gate	Attend le verdict de SonarQube (webhook) et arrête le pipeline s'il échoue	6 s
Build image Docker	Image eclipse-temurin 17 JRE, utilisateur non-root, healthcheck Actuator	11 s
Push Docker Hub	Publie dajbinary/autoloc-api:<build>-<commit> et :latest	122 s

** Durées mesurées sur le build n°3, entièrement au vert (image 3-6faaca5, 120 Mo).*

Fichiers ajoutés au dépôt

Fichier	Contenu
Jenkinsfile	Pipeline déclaratif : 7 étapes, déclenchement par scrutation de GitHub toutes les 5 minutes, nettoyage du conteneur MySQL et déconnexion de Docker Hub en fin de build
ci/docker-compose.yml	Jenkins, SonarQube et sa base PostgreSQL, sur le réseau autoloc-ci
ci/jenkins/	Image Jenkins avec Docker CLI, liste des plugins et configuration as code (casc.yaml) : compte admin, serveur SonarQube, identifiants et job créés au démarrage
Dockerfile, .dockerignore	Image de l'application à partir du jar construit par le pipeline
ci/.env.example	Modèle des secrets ; le vrai fichier ci/.env est exclu de Git

Bonnes pratiques appliquées

- **Infrastructure as code** : aucune configuration manuelle dans l'interface Jenkins, tout est versionné ; l'environnement se reconstruit à l'identique.
- **Secrets hors du dépôt** : mots de passe générés aléatoirement, jetons SonarQube et Docker Hub stockés comme identifiants Jenkins, jamais affichés dans les logs.
- **Base de test isolée** : chaque build crée sa propre base MySQL et la supprime à la fin, même en cas d'échec.
- **Quality Gate bloquant** : une image n'est publiée que si les tests et la qualité sont validés.
- **Traçabilité** : chaque image est étiquetée avec le numéro de build et le commit Git.

Problèmes rencontrés et solutions

Problème	Cause	Solution
Jenkins ne pouvait pas lancer ./mvnw	Fichier non exécutable dans Git (créé sous Windows)	Mode exécutable enregistré dans Git
Build n°1 : échec du checkout GitHub	Le DNS du partage de connexion ne renvoyait qu'une adresse IPv6 (NAT64), inaccessible aux conteneurs	Résolveurs DNS publics pour le conteneur Jenkins
Build n°2 : Quality Gate bloqué	SonarQube refuse par défaut les webhooks vers une adresse privée	sonar.validateWebhooks=false sur ce réseau local
Healthcheck SonarQube en échec	wget absent de l'image SonarQube	Utilisation de curl

Lancer l'environnement

```
docker compose -f ci/docker-compose.yml --env-file ci/.env up -d
Jenkins : http://localhost:8090    SonarQube : http://localhost:9000
```

9. Bilan et perspectives

Le projet livre une application complète et testée, dont la qualité est mesurée en continu et dont chaque version validée est publiée automatiquement sous forme d'image Docker.

9	54	283	91,4 %	A
entités	pages	tests	couverture	notes Sonar

Pistes d'amélioration

- **Déploiement continu** : ajouter une étape qui déploie l'image publiée avec sa base MySQL (Docker Compose ou Kubernetes) puis vérifie /actuator/health.
- **Déclenchement par webhook GitHub** au lieu de la scrutation toutes les 5 minutes, via une URL publique (tunnel ou serveur Jenkins hébergé).
- **Agent Jenkins dédié** pour les builds Docker, plutôt que le contrôleur Jenkins exécuté en root avec le socket Docker de l'hôte : acceptable pour une CI locale, à éviter en production.
- **Migrations de schéma** versionnées (Flyway ou Liquibase) à la place de la mise à jour automatique par Hibernate.
- **Analyse de vulnérabilités de l'image** (Trivy ou Docker Scout) avant le push.